

Lecture 6

간단한 아케이드 게임의 전투와 게임 관리

Unity를 이용한 2D 게임프로그래밍

강영민

동명대학교 게임학부

목차

- 1 캐릭터에 콜라이더 설정
- 2 Rigidbody 2D와 물리
- 3 데미지 처리 – Health & DamageDealer
- 4 발사체 제작
- 5 Layer와 충돌 제어
- 6 플레이어 슈팅
- 7 Input 시스템 – 발사 제어
- 8 Player.cs – CallbackContext 버전
- 9 정리

캐릭터에 콜라이더 설정 – Prefab 폴더

Prefab 폴더 확인

- 모든 캐릭터 Prefab에 **PathFinder** 스크립트가 설정되어야 함
- 다수의 적 비행체 프리팹 생성한 상태
- 동시에 동일한 컴포넌트를 넣고 싶으면 **다중 선택**하여 컴포넌트 추가

Tip: 다중 선택 편집

여러 Prefab을 선택한 뒤 Inspector에서 컴포넌트를 추가하면 한꺼번에 적용된다.

캐릭터에 콜라이더 설정 – Trigger 설정

● Circle Collider 2D 설정

- 캐릭터에 **Circle Collider 2D** 추가
- **Is Trigger** 체크 → 물리적 충돌 대신 트리거 이벤트 발생

i Collider vs Trigger

- **Collider**: 물리적 충돌 – 물체가 서로 밀어냄
- **Trigger**: 겹침 감지 – 물리적 반응 없이 이벤트만 발생
→ `OnTriggerEnter2D()` 콜백 호출

Unity 2D 충돌 감지 시스템

충돌 감지의 두 가지 조건

Unity에서 두 오브젝트의 충돌을 감지하려면 **두 가지**가 모두 필요하다: (1) 양쪽 모두 **Collider 2D** 보유 (2) 최소 한쪽에 **Rigidbody 2D** 보유

★ Collider 2D 종류와 용도

종류	형태	용도
Box Collider 2D	사각형	상자, 벽
Circle Collider 2D	원형	캐릭터, 탄환
Capsule Collider 2D	캡슐형	레이저
Polygon Collider 2D	다각형	불규칙 형태

콜백 함수 비교

Collider (Is Trigger 해제): OnCollisionEnter2D / Stay2D / Exit2D

Trigger (Is Trigger 체크): OnTriggerEnter2D / Stay2D / Exit2D

Rigidbody 2D 추가

Rigidbody 2D 컴포넌트

- 모든 캐릭터 Prefab에 **Rigidbody 2D** 추가
- Body Type 변경: **Dynamic** → **Kinematic**

주의: Body Type

Dynamic으로 두면 중력의 영향을 받아 아래로 떨어진다.
스크립트로 위치를 제어할 경우 반드시 **Kinematic**으로 설정!

Dynamic vs. Kinematic

Dynamics (동역학)

- 물리 시뮬레이션에 의해 힘이 계산되고 그 힘에 의해 물체가 가속/이동
- 물체의 위치를 명시적으로 설정하지 않고 위치를 결정하는 조건을 지정
- 물리 법칙에 따라 움직임 (자연스러운 물리 시뮬레이션)
- 제어의 어려움 존재

Kinematics (운동학)

- 물체의 움직임을 표현하는 곡선을 다루는 분야
- 물체의 위치를 **명시적으로 설정**
- 제어가 용이함
- 게임에서 스크립트로 직접 위치를 제어할 때 적합

Rigidbody 2D – Body Type 비교

세 가지 Body Type 비교

속성	Dynamic	Kinematic
물리 시뮬레이션	중력, 힘, 토크 적용	적용 안됨
이동 방식	AddForce, velocity	transform.position 직접 설정
다른 오브젝트와	서로 밀어냄	밀리지 않음
충돌 감지	가능	가능 (Trigger 권장)
적합한 용도	물리 퍼즐, 던지기	적/플레이어 이동 제어

★ Static Body Type

Static: 절대 움직이지 않는 오브젝트 (벽, 바닥). Rigidbody 2D 없이 Collider만 있어도 Static으로 동작하지만, 명시적으로 설정하면 **성능 최적화**에 유리하다.

데미지를 처리하자 - Health.cs

 Health.cs

```
1 using UnityEngine;
2
3 public class Health : MonoBehaviour
4 {
5     [SerializeField] int health = 50;
6
7     void OnTriggerEnter2D(Collider2D other)
8     {
9         DamageDealer damageDealer =
10             other.GetComponent<DamageDealer>();
11         if (damageDealer != null)
12         {
13             TakeDamage(damageDealer.GetDamage());
14             damageDealer.Hit();
15         }
16     }
17
18     void TakeDamage(int damage)
19     {
20         health -= damage;
21         if (health < 0)
22         {
23             Destroy(gameObject);
24         }
25     }
26 }
27
```

데미지를 처리하자 – DamageDealer.cs

DamageDealer.cs

```
1 using UnityEngine;
2
3 public class DamageDealer : MonoBehaviour
4 {
5     [SerializeField] int damage = 10;
6
7     public int GetDamage()
8     {
9         return damage;
10    }
11
12    public void Hit()
13    {
14        Destroy(gameObject);
15    }
16 }
17
```

역할 분담

Health: 체력 관리, 트리거 충돌 시 데미지 처리 **DamageDealer:** 데미지 값 제공, 충돌 후 자기 자신 파괴

🎯 적기에 Damage Dealer 추가

- 모든 적 Prefab에 **DamageDealer** 컴포넌트 추가
- 적기가 충돌 시 플레이어에게 데미지를 입힘

❤️ 아군기에 Health 추가

- Player Prefab에 **Health** 컴포넌트 추가
- 기본 체력값: 50

★ 적기와 아군기의 충돌 결과

- 적기가 가진 **damage**만큼 아군기 **health** 감소
- 적기는 즉시 파괴 – **Health.cs**에서 호출하는 **DamageDealer.Hit()**에 의해
- 아군기는 **health**가 0보다 작아지면 파괴

컴포넌트 기반 설계 패턴

Component Pattern이란?

Unity는 **상속**이 아닌 **컴포넌트 조합**으로 게임 오브젝트의 동작을 구성한다.

- 각 스크립트는 **하나의 책임**만 담당 (단일 책임 원칙)
- 필요한 기능을 조합하여 다양한 오브젝트를 만들 수 있음

GetComponent<T>()

다른 컴포넌트에 접근하는 핵심 메서드

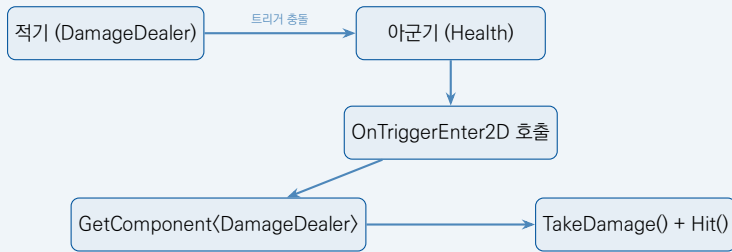
- 같은 GameObject의 컴포넌트 검색
- 없으면 null 반환
- 반드시 **null 체크** 필요

이 강의의 컴포넌트 구성

Health	체력, 피해 수신
DamageDealer	데미지 전달
Shooter	발사 로직
Player	입력 처리
PathFinder	경로 이동

OnTriggerEnter2D 동작 원리

⇄ 충돌 시 콜백 흐름



⚠ 주의: other 파라미터

OnTriggerEnter2D(Collider2D other)에서 other는 **상대방의 Collider**이다.
따라서 other.GetComponent<DamageDealer>()는 **충돌한 상대에서** 컴포넌트를 찾는다.

발사체 제작 – Player 발사체

Player 발사체 Prefab 만들기

- 1 **Empty 객체** 생성 → 이름: Projectile_player
- 2 레이저 스프라이트를 자식으로 배치
- 3 **DamageDealer** 컴포넌트 추가 (Damage: 10)
- 4 Scale: X=2, Y=2, Z=2

발사체에 콜라이더 설정

- **Capsule Collider 2D** 추가
- **Is Trigger** 체크
- Direction: **Vertical**

발사체 제작 – Enemy 발사체

Enemy 발사체 만들기

- 플레이어 발사체를 **복사**하여 제작
- 스프라이트를 적 레이저로 변경 (예: 빨간색 레이저)
- 이름: `Projectile_enemy`

문제점: 플레이어 발사체가 플레이어와 반응

- 발사체(DamageDealer) + 플레이어(Health) → 자기 발사체에 자신이 맞음!
- 컴포넌트 구성 정리:
 - 발사체 – DamageDealer
 - Player – Health
 - Enemy – DamageDealer + Health

발사체를 프리팹으로

Prefab으로 저장

- 완성된 발사체를 **Prefabs** 폴더에 끌어다 놓기
- Hierarchy에서 원본 삭제
- Prefab으로 관리하면 런타임에 `Instantiate()`로 생성 가능

Instantiate와 Destroy – 오브젝트 생명주기

🔄 런타임 오브젝트 생성/파괴

메서드	설명
Instantiate(prefab, pos, rot)	Prefab의 복제본(인스턴스) 을 생성하여 씬에 배치
Destroy(obj)	현재 프레임 끝에 오브젝트 파괴
Destroy(obj, time)	time초 후에 오브젝트 파괴 (지연 파괴)

★ Prefab → Instance

Prefab: 에디터에서 만든 **틀(템플릿)**

Instance: 런타임에 생성된 **실체(복제본)**

Prefab 자체는 씬에 존재하지 않으며,
Instantiate()로 만든 인스턴스만 게임에 등장한다.

⚠ 메모리 누수 주의

발사체를 생성만 하고 파괴하지 않으면
씬에 오브젝트가 계속 쌓여 **성능 저하** 발생.

반드시 Destroy(missile, lifeTime)으로
일정 시간 후 자동 파괴 설정!

Layer 만들기

- **Player** 레이어 생성 (User Layer 8)
- **Enemy** 레이어 생성 (User Layer 9)
- 플레이어와 플레이어 발사체 → **Player** 레이어
- 적과 적 발사체 → **Enemy** 레이어

주의

Layer를 추가한 이후에는 각 Prefab에서 직접 **레이어를 지정**해야 한다.

Layer Collision Matrix

☐ Edit → Project Settings → Physics 2D

- **Layer Collision Matrix**에서 같은 레이어 간 충돌을 비활성화
- Player-Player: **비활성** (자기 발사체와 충돌 방지)
- Enemy-Enemy: **비활성** (적 발사체와 적 충돌 방지)
- Player-Enemy: **활성** (전투 충돌 허용)

★ 핵심 포인트

같은 팀의 오브젝트끼리 충돌하지 않도록 레이어를 분리하는 것이 슈팅 게임의 기본 패턴이다.

Layer Overrides

Player 측 설정

Collider 2D의 **Layer Overrides**:

- Contact Capture Layers: **Enemy**
- Callback Layers: **Enemy**

→ Enemy 레이어와만 충돌 감지

Enemy 측 설정

Collider 2D의 **Layer Overrides**:

- Contact Capture Layers: **Player**
- Callback Layers: **Player**

→ Player 레이어와만 충돌 감지

Enemies – DamageDealer & Health 추가

적에게도 **Health** 컴포넌트를 추가하여 플레이어 발사체에 맞으면 파괴되도록 한다.

Unity Layer 시스템의 구조

Layer 시스템 이해

- Unity는 **32개의 레이어**를 지원 (0-31 번)
- 내부적으로 **32비트 비트마스크**로 관리
- 0-7번: 빌트인 레이어 (Default, TransparentFX, Ignore Raycast, Water, UI 등)
- 8-31번: **사용자 정의 레이어** (User Layer)

★ Collision Matrix vs Layer Overrides

방법	특징
Collision Matrix	프로젝트 전역 설정. 레이어 쌍 단위로 충돌 on/off
Layer Overrides	개별 Collider 단위 설정. 특정 오브젝트만 세밀 제어

두 가지를 **조합**: Collision Matrix로 큰 틀, Layer Overrides로 세부 조정.

Coroutine – 핵심 개념

🕒 Coroutine이란?

코루틴은 실행을 일시 중단(yield)했다가 다음 프레임 또는 일정 시간 후 재개할 수 있는 특수한 메서드이다. 반환형: `IEnumerator` / `yield return`으로 중단점 지정 / 게임 루프를 멈추지 않으면서 시간 지연 동작 구현 가능

★ 주요 yield 구문

구문	동작
<code>yield return null</code>	다음 프레임까지 대기
<code>yield return new WaitForSeconds(t)</code>	t초 동안 대기
<code>yield break</code>	코루틴 즉시 종료

💡 StartCoroutine / StopCoroutine

`StartCoroutine()`으로 시작, `StopCoroutine()`으로 외부에서 중지 가능. 참조를 `Coroutine` 변수에 저장해야 나중에 중지할 수 있다.

플레이어 슈팅 – Player.cs 수정

슈팅 구조

실제 슈팅 동작은 **Shooter.cs**가 담당하고, **Player.cs**는 Shooter 컴포넌트를 참조하여 **isFiring** 플래그만 제어한다.

Player.cs – 슈팅 관련 코드 (기본)

```
1  [SerializeField] GameObject projectilePrefab;
2  Shooter shooter;
3
4  void Awake()
5  {
6      shooter = GetComponent<Shooter>();
7  }
8
9  public void OnAttack(InputValue value)
10 {
11     if (shooter != null)
12     {
13         shooter.isFiring = value.isPressed;
14     }
15 }
16
```

Shooter.cs – 필드 및 Fire 제어

Shooter.cs (Part 1)

```
1 using System.Collections;
2 using UnityEngine;
3
4 public class Shooter : MonoBehaviour
5 {
6     [SerializeField] GameObject projectilePrefab;
7     [SerializeField] float projectileSpeed = 10f;
8     [SerializeField] float projectileLifeTime = 5f;
9     [SerializeField] float firingRate = 0.2f;
10    public bool isFiring;
11    Coroutine firingRoutine;
12
13    void Update() { Fire(); }
14
15    void Fire()
16    {
17        if (isFiring && firingRoutine == null)
18        {
19            firingRoutine = StartCoroutine(FireContinuously());
20        }
21        else if (!isFiring && firingRoutine != null)
22        {
23            StopCoroutine(firingRoutine);
24            firingRoutine = null;
25        }
26    }
27 }
```


Shooter.cs – FireContinuously

Shooter.cs (Part 2)

```
1  IEnumerator FireContinuously()
2  {
3      while (true)
4      {
5          GameObject missile = Instantiate(
6              projectilePrefab,
7              transform.position,
8              Quaternion.identity);
9
10         Rigidbody2D rb =
11             missile.GetComponent<Rigidbody2D>();
12         if (rb != null)
13         {
14             rb.linearVelocity =
15                 transform.up * projectileSpeed;
16         }
17
18         Destroy(missile, projectileLifeTime);
19         yield return new WaitForSeconds(firingRate);
20     }
21 }
22 }
23
```

★ Shooter 동작 요약

- 1 Player.cs에서 `isFiring = true` 설정
- 2 `Update()`에서 `Fire()` 호출
- 3 `isFiring`이 `true`이고 `firingRoutine`이 `null`이면
→ `StartCoroutine(FireContinuously())` 시작
- 4 코루틴 내부에서 무한 루프로 발사체를 생성
 - `Instantiate()`로 발사체 Prefab 생성
 - `Rigidbody2D.linearVelocity`로 위쪽 방향 속도 부여
 - `Destroy()`로 일정 시간 후 자동 파괴
 - `WaitForSeconds(firingRate)`로 발사 간격 제어
- 5 `isFiring`이 `false`가 되면 코루틴 중지

Rigidbody2D.linearVelocity – 발사체 이동

velocity를 이용한 이동

Rigidbody2D.linearVelocity: 물체의 **선형 속도 벡터**를 직접 설정
transform.up: 오브젝트의 **로컬 위쪽** 방향 (기본: Vector2(0, 1))
transform.up * speed: 위쪽 방향으로 speed만큼의 속도

플레이어 발사체

transform.up = (0, 1) → 위쪽으로 발사

적 발사체

적기는 아래를 향하도록 회전 필요
또는 **-transform.up** 사용

velocity vs transform.position 이동

velocity: Rigidbody가 매 프레임 자동으로 위치 갱신 → 물리 충돌 정상 감지
transform.position: 직접 이동 → 빠른 오브젝트에서 충돌 누락 가능

미사일 발사 – 문제점: 발사 중지

⚠ 문제: 발사를 멈출 수 없다

OnAttack(InputValue value) 사용 시, 버튼이 눌릴 때만 호출된다.
→ 버튼을 놓았을 때의 이벤트를 감지할 수 없음

💡 해결 방법

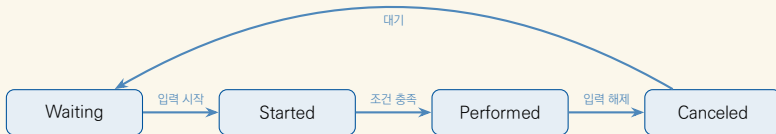
- 1 Input Action에서 Attack 액션의 **Interactions**에 **Press** 추가
- 2 Trigger Behavior: **Press And Release**로 변경
- 3 InputAction.CallbackContext를 사용하여 performed/canceled 구분

New Input System – 아키텍처

🏗️ Player Input 컴포넌트의 Behavior 모드

모드	특징
Send Messages	OnMove(InputValue) 형태로 호출. 간단하지만 performed/canceled 구분 불가
Invoke Unity Events	Inspector에서 이벤트 연결. CallbackContext로 상태 구분 가능
Invoke C# Events	코드에서 직접 이벤트 구독. 가장 유연하지만 설정이 복잡

★ InputAction 상태 머신



Attack 액션 설정

- Input Action Asset 열기
- **Attack** 액션 선택
- Action Type: **Button**
- Interactions에 **Press** 추가
- Trigger Behavior: **Press And Release**

Press And Release

Press And Release로 설정하면 버튼을 누를 때(performed)와 놓을 때(canceled) 모두 이벤트가 발생한다.

Invoke Unity Events 사용

⚡ Player Input 컴포넌트 설정

- Behavior: **Invoke Unity Events**
- Events → Player:
 - Move (CallbackContext) → `Player.OnMove`
 - Attack (CallbackContext) → `Player.OnAttack`

★ CallbackContext vs InputValue

Invoke Unity Events 모드에서는 `InputAction.CallbackContext`를 파라미터로 받아 `performed`, `canceled` 상태를 직접 확인할 수 있다.

Send Messages vs Invoke Unity Events – 코드 비교

✉ Send Messages

파라미터: InputValue

- `value.Get<Vector2>()` – 값 읽기
- `value.isPressed` – 눌림 여부
- 버튼 누를 때만 호출
- 놓는 시점 감지 불가

장점: 코드가 간결

단점: Press/Release 구분 어려움

⚡ Invoke Unity Events

파라미터: CallbackContext

- `context.ReadValue<T>()` – 값 읽기
- `context.performed` – 눌림
- `context.canceled` – 놓음
- 양쪽 모두 콜백 수신

장점: 상태별 세밀한 제어

단점: 코드가 약간 복잡

⚠ 혼용 금지

한 프로젝트에서 두 모드를 섞으면 혼란이 생긴다. Player Input의 Behavior를 변경하면 모든 콜백의 **파라미터 타입**이 바뀌므로 주의!

Player.cs – 전체 코드 (CallbackContext)

Player.cs (Part 1: 필드 & 초기화)

```
1 using UnityEngine;
2 using UnityEngine.InputSystem;
3
4 public class Player : MonoBehaviour
5 {
6     Vector2 moveInput = new Vector2(0f, 0f);
7     private float speed = 5.0f;
8
9     Vector2 minBounds;
10    Vector2 maxBounds;
11
12    void Start()
13    {
14        Camera cam = Camera.main;
15        minBounds = cam.ViewportToWorldPoint(
16            new Vector2(0, 0));
17        maxBounds = cam.ViewportToWorldPoint(
18            new Vector2(1, 1));
19    }
20 }
```

Player.cs – Update & OnMove



Player.cs (Part 2: Update & OnMove)

```
1  void Update()
2  {
3      Vector2 moveDelta = moveInput * speed * Time.deltaTime;
4      Vector2 newPosition = (Vector2)transform.position + moveDelta;
5      newPosition.x = Mathf.Clamp(newPosition.x, minBounds.x, maxBounds.x);
6      newPosition.y = Mathf.Clamp(newPosition.y, minBounds.y, maxBounds.y);
7      transform.position = newPosition;
8  }
9
10 public void OnMove(InputAction.CallbackContext context)
11 {
12     if (context.performed)
13     {
14         moveInput = context.ReadValue<Vector2>();
15     }
16     else if (context.canceled)
17     {
18         moveInput = new Vector2(0f, 0f);
19     }
20 }
21
```

Player.cs – OnAttack (발사 제어)

Player.cs (Part 3: OnAttack)

```
1  Shooter shooter;
2
3  void Awake()
4  {
5      shooter = GetComponent<Shooter>();
6  }
7
8  public void OnAttack(
9      InputAction.CallbackContext context)
10 {
11     if (context.performed)
12     {
13         Debug.Log("Attack started");
14         shooter.isFiring = true;
15     }
16     else if (context.canceled)
17     {
18         Debug.Log("Attack stopped");
19         shooter.isFiring = false;
20     }
21 }
22 }
23
```

★ CallbackContext 사용 핵심

- `context.performed` - 버튼이 눌렸을 때
 - `shooter.isFiring = true`
- `context.canceled` - 버튼을 놓았을 때
 - `shooter.isFiring = false`

💡 OnMove도 동일한 패턴

OnMove에서도 `context.performed`일 때 입력값을 읽고,
`context.canceled`일 때 입력을 0으로 초기화한다.

📖 이번 강의 요약

- ❶ **콜라이더 & 트리거**: 캐릭터에 Collider 2D + Is Trigger 설정
- ❷ **Rigidbody 2D**: Kinematic으로 설정하여 스크립트 제어
- ❸ **Health / DamageDealer**: 데미지 처리를 위한 컴포넌트 설계
- ❹ **발사체**: Prefab으로 만들어 Instantiate로 생성
- ❺ **Layer & Collision Matrix**: 같은 팀 충돌 방지
- ❻ **Layer Overrides**: 특정 레이어와만 충돌 감지
- ❼ **Shooter.cs**: 코루틴을 이용한 연속 발사 구현
- ❽ **Input System**: CallbackContext로 Press/Release 감지

?

적도 발사하게 해 보라

감사합니다